

Competitive Programming

Section 29 Editorial: Modular Arithmetic

Competitive Programming Course

2026-07-06

Table of contents

1	Editorial overview	3
2	A. Power Queries	4
2.1	Restatement	4
2.2	Constraints and traps	4
2.3	Algorithm	4
2.4	Correctness proof	4
2.5	Complexity	4
2.6	Full C++ solution	5
2.7	Full Python solution	5
2.8	Common wrong answers and tests	6
3	B. Fraction Queries	7
3.1	Restatement	7
3.2	Observations	7
3.3	Algorithm	7
3.4	Correctness proof	7
3.5	Complexity	7
3.6	Full C++ solution	8
3.7	Full Python solution	8
3.8	Common wrong answers and tests	9
4	C. Affine Repeater	10
4.1	Restatement	10
4.2	Observations	10
4.3	Algorithm	10
4.4	Correctness proof	11
4.5	Complexity	11
4.6	Full C++ solution	11
4.7	Full Python solution	12
4.8	Common wrong answers and tests	13
5	Testing discussion	14
6	What to carry forward	15

1 Editorial overview

Problem	Tool	Main risk
A. Power Queries	binary exponentiation	large exponent
B. Fraction Queries	Fermat inverse	division only valid when inverse exists
C. Affine Repeater	power plus geometric sum	<code>a == 1</code> and negative C++ residues

2 A. Power Queries

2.1 Restatement

For each query, compute $a^b \bmod m$.

2.2 Constraints and traps

b can be as large as 10^{18} , so repeated multiplication is impossible. The modulus can be 1, in which case every answer is 0.

2.3 Algorithm

Use binary exponentiation:

```
result = 1 mod m
a = a mod m
while b > 0:
    if b is odd, result = result * a mod m
    a = a * a mod m
    b = b / 2
```

2.4 Correctness proof

At every step, the algorithm has split the original power into two parts: the power already multiplied into `result`, and the remaining power represented by `baseb`. If the current exponent bit is 1, multiplying by `base` accounts for that bit. Squaring `base` moves to the next binary place, and halving `b` discards the processed bit. When `b` becomes zero, every binary bit has been accounted for, so `result == aoriginal_b mod m`. $\square$

2.5 Complexity

```
Time:  $O(q \log b)$ 
Memory:  $O(1)$ 
```

2.6 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

long long mod_pow(long long a, long long b, long long mod) {
    a %= mod;
    long long result = 1 % mod;
    while (b > 0) {
        if (b & 1LL) result = (__int128)result * a % mod;
        a = (__int128)a * a % mod;
        b >>= 1LL;
    }
    return result;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int q;
    if (!(cin >> q)) return 0;
    while (q--) {
        long long a, b, mod;
        cin >> a >> b >> mod;
        cout << mod_pow(a, b, mod) << '\n';
    }
    return 0;
}
```

2.7 Full Python solution

```
import sys

def mod_pow(a: int, b: int, mod: int) -> int:
    a %= mod
    result = 1 % mod
    while b:
        if b & 1:
            result = result * a % mod
        a = a * a % mod
        b >>= 1
    return result

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
```

```

if not data:
    return
q = data[0]
out = []
idx = 1
for _ in range(q):
    a, b, mod = data[idx], data[idx + 1], data[idx + 2]
    idx += 3
    out.append(str(mod_pow(a, b, mod)))
print("\n".join(out))

if __name__ == "__main__":
    main()

```

2.8 Common wrong answers and tests

O(b) multiplication: fails huge b.
 Initializing result = 1 when mod = 1: should become 0.
 Overflowing multiplication in harder variants.

3 B. Fraction Queries

3.1 Restatement

For prime p , compute $a / b \bmod p$ for many queries. The input guarantees $b \bmod p \neq 0$.

3.2 Observations

Division means multiplying by an inverse:

$$a / b == a * b^{-1} \pmod{p}$$

Since p is prime and b is not divisible by p , Fermat's little theorem gives:

$$b^{-1} == b^{p-2} \pmod{p}$$

3.3 Algorithm

```
for each query:
    inv = pow_mod(b, p-2, p)
    print (a mod p) * inv mod p
```

3.4 Correctness proof

Because p is prime and b is not divisible by p , Fermat's little theorem states $b^{p-1} == 1 \pmod{p}$. Multiplying both sides by the formal inverse meaning gives $b * b^{p-2} == 1 \pmod{p}$, so b^{p-2} is the modular inverse of b . Multiplying a by this inverse is exactly modular division. $\square$

3.5 Complexity

```
Time:  $O(q \log p)$ 
Memory:  $O(1)$ 
```

3.6 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

long long mod_pow(long long a, long long b, long long mod) {
    a %= mod;
    long long result = 1 % mod;
    while (b > 0) {
        if (b & 1LL) result = (__int128)result * a % mod;
        a = (__int128)a * a % mod;
        b >>= 1LL;
    }
    return result;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    long long p;
    int q;
    if (!(cin >> p >> q)) return 0;
    while (q--) {
        long long a, b;
        cin >> a >> b;
        long long inv = mod_pow(b, p - 2, p);
        long long answer = (__int128)(a % p) * inv % p;
        cout << answer << '\n';
    }
    return 0;
}
```

3.7 Full Python solution

```
import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    p, q = data[0], data[1]
    idx = 2
    out = []
    for _ in range(q):
        a, b = data[idx], data[idx + 1]
        idx += 2
```



```
        out.append(str((a % p) * pow(b % p, p - 2, p) % p))
    print("\n".join(out))
```

```
if __name__ == "__main__":
    main()
```

3.8 Common wrong answers and tests

Using a / b as floating-point division.
Using Fermat inverse with composite modulus.
Forgetting to reduce b before exponentiation.
Testing $p = 2$.

4 C. Affine Repeater

4.1 Restatement

For each query, repeatedly apply:

$$f(x) = a*x + b$$

exactly k times modulo 1,000,000,007.

4.2 Observations

Expanding the first few applications:

$$\begin{aligned} f^1(x) &= a*x + b \\ f^2(x) &= a^2*x + a*b + b \\ f^3(x) &= a^3*x + a^2*b + a*b + b \end{aligned}$$

Therefore:

$$f^k(x) = a^k*x + b*(1 + a + \dots + a^{(k-1)})$$

The sum is k when $a == 1$. Otherwise:

$$1 + a + \dots + a^{(k-1)} = (a^k - 1) / (a - 1)$$

Modulo a prime, the division is multiplication by inverse.

4.3 Algorithm

```
reduce a, b, x modulo MOD
if k == 0: answer is x
ak = pow_mod(a, k, MOD)
if a == 1:
    geom = k mod MOD
else:
    geom = (ak - 1) * inverse(a - 1) mod MOD
answer = ak*x + b*geom mod MOD
```

4.4 Correctness proof

The expansion can be proven by induction. For $k = 0$, the value is x . Assume after k applications the formula is true. Applying f once more gives:

$$\begin{aligned} & a \cdot (a^k x + b(1 + a + \dots + a^{k-1})) + b \\ &= a^{k+1} x + b(a + a^2 + \dots + a^k + 1) \end{aligned}$$

which is the same formula for $k+1$. The geometric sum formula is valid when $a \neq 1$; when $a = 1$, every term is 1, so the sum is k . Therefore the algorithm computes exactly the repeated transformation modulo MOD. $\square$

4.5 Complexity

Time: $O(q \log k)$
Memory: $O(1)$

4.6 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

const long long MOD = 1000000007LL;

long long mod_pow(long long a, long long b) {
    a %= MOD;
    if (a < 0) a += MOD;
    long long result = 1;
    while (b > 0) {
        if (b & 1LL) result = result * a % MOD;
        a = a * a % MOD;
        b >>= 1LL;
    }
    return result;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int q;
    if (!(cin >> q)) return 0;
    while (q--) {
        long long a, b, x, k;
        cin >> a >> b >> x >> k;
        a %= MOD;
```

```

    b %= MOD;
    x %= MOD;
    if (k == 0) {
        cout << x << '\n';
        continue;
    }
    long long ak = mod_pow(a, k);
    long long geom;
    if (a == 1) {
        geom = k % MOD;
    } else {
        geom = (ak - 1 + MOD) % MOD * mod_pow(a - 1, MOD - 2) % MOD;
    }
    cout << (ak * x + b * geom) % MOD << '\n';
}
return 0;
}

```

4.7 Full Python solution

```

import sys

MOD = 1_000_000_007

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    q = data[0]
    idx = 1
    out = []
    for _ in range(q):
        a, b, x, k = data[idx], data[idx + 1], data[idx + 2], data[idx + 3]
        idx += 4
        a %= MOD
        b %= MOD
        x %= MOD
        if k == 0:
            out.append(str(x))
            continue
        ak = pow(a, k, MOD)
        if a == 1:
            geom = k % MOD
        else:
            geom = (ak - 1) * pow(a - 1, MOD - 2, MOD) % MOD
        out.append(str((ak * x + b * geom) % MOD))

```

```
print("\n".join(out))

if __name__ == "__main__":
    main()
```

4.8 Common wrong answers and tests

Simulating k times.
Checking original $a == 1$ instead of reduced $a == 1$.
Forgetting $k == 0$.
C++ inverse of a^{-1} when $a = 0$ needs negative normalization.

5 Testing discussion

The random generator for the affine problem brute-forces small k and uses the formula for large k . When writing your own stress tests, compare:

```
slow simulation for  $k \leq 20$   
fast formula for the same random cases
```

6 What to carry forward

Modular arithmetic is not just “put % MOD everywhere.” Carry forward these rules:

```
fast powers are  $O(\log \text{exponent})$   
division requires an inverse  
Fermat inverse requires a prime modulus  
C++ negative residues must be normalized  
special cases are part of the math
```